

UML-діаграми проєкту

Remote File Folder Client

(спрощений аналог Google Drive)

Уточкіна Яна

Етап: Проєктування архітектури та поведінки системи
12 вересня 2026 р.

Анотація

У документі представлено комплект UML-діаграм для проєкту **Remote File Folder Client** — застосунку для роботи з віддаленою файловою текою (спрощений аналог Google Drive) з веб- та десктоп-клієнтами, Go-бекендом, PostgreSQL та синхронізацією файлів за принципом diff-маніфестів. Діаграми впорядковано відповідно до життєвого циклу проєктування: від аналізу вимог і високорівневої архітектури до деталізованого проєктування окремих підсистем. Усі діаграми побудовано в нотатції Mermaid.

Зміст

1	Аналіз вимог	3
2	Архітектурне проєктування (високий рівень)	3
2.1	Діаграма прецедентів	3
2.2	Діаграма класів (загальна структура)	5
2.3	Діаграма розгортання (загальна архітектура)	8
3	Деталізоване проєктування	9
3.1	VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”	10
3.2	Діаграми взаємодії (Sequence Diagrams)	12
3.3	Діаграма станів	18
3.4	Діаграма компонентів	20
3.5	Діаграма активності	21

1. Аналіз вимог

Система надає користувачу автентифікований доступ до персональної віддаленої теки (“віртуального диска”), у якій він може переглядати список файлів разом з їх атрибутами (ім’я, дата створення, дата модифікації, автор вивантаження, автор останньої редакції), вивантажувати, завантажувати та видаляти файли, а також синхронізувати обраний локальний каталог із віддаленим.

Ключові функціональні вимоги:

- автентифікація користувача на віддаленому сервері;
- перегляд атрибутів файлів із можливістю показу/приховування стовпців (стовпець імені файлу приховати не можна);
- перегляд вмісту файлів окремих типів: `.java` – як текст, `.png` – як зображення;
- сортування файлів за іменем користувача, який востаннє редагував файл (за зростанням / спаданням);
- фільтрація файлів: усі файли / лише `.cs` / лише `.jpg`;
- синхронізація локальної теки з віддаленою на основі порівняння маніфестів (`name`, `size`, `mtime`, `sha256`) та обробка конфліктів редагування.

Архітектурно система складається з React-клієнта (веб), Swift-клієнта (macOS, з повноцінною фоновією синхронізацією через FSEvents), Go API-сервера (шари `handler` / `service` / `repository`), PostgreSQL для метаданих та абстрактного сховища файлів (локальний диск або S3-сумісне сховище), розгорнутих на домашньому Ubuntu-сервері та опублікованих через Cloudflare Tunnel.

На основі цих вимог у наступних розділах побудовано діаграму прецедентів, а також архітектурні та деталізовані діаграми, що описують структуру і поведінку системи.

2. Архітектурне проєктування (високий рівень)

На цьому етапі визначається загальна архітектура системи: хто є її користувачами і які сценарії вона підтримує (діаграма прецедентів), яка загальна структура даних (діаграма класів високого рівня) та яка фізична топологія розгортання (діаграма розгортання).

2.1 Діаграма прецедентів

Діаграма (рис. 1) показує єдиного актора *Користувача*, який працює із системою через веб- або десктоп-клієнт, та основні прецеденти, що відповідають функціональним вимогам розділу 1. Частина операцій над файлами включають («include») прецедент “Автентифікуватися”, оскільки до-

ступ до робочої області можливий лише після успішного входу. Перегляд вмісту файлу *розширюється* («extend») двома альтернативними сценаріями відображення (. java / . png), а синхронізація – сценарієм розв’язання конфлікту, що активується лише за наявності розбіжностей.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми прецедентів, тому актора, прецеденти та межу системи побудовано засобами flowchart (актор – прямокутник зі заокругленими кутами, прецеденти – “stadium”-форма, межа системи – subgraph), із явним підписом стереотипів «include»/«extend» на зв’язках.

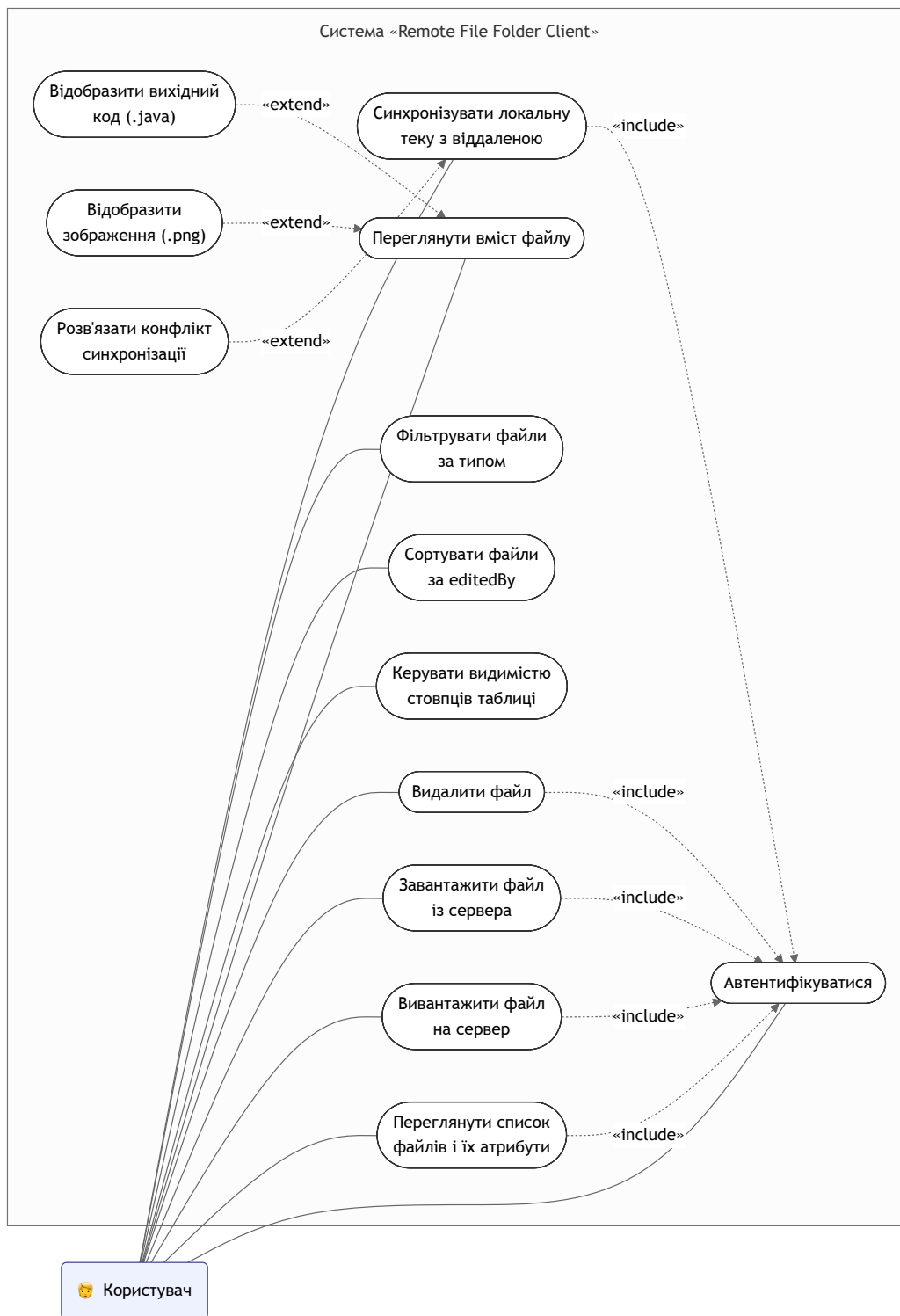


Рис. 1: Діаграма прецедентів системи Remote File Folder Client

2.2 Діаграма класів (загальна структура)

Рис. 2 відображає загальну структуру даних та основні взаємозв'язки між доменними класами системи, без деталізації методів окремих сценаріїв: сутності User та File, сервісний шар (AuthService, FileService, SyncDiffService),

шар доступу до даних (FileRepository) та абстракцію сховища файлів FileStorage, реалізовану локальним сховищем на диску та S3-сумісним сховищем – відповідно до принципу “сховище за інтерфейсом”, закладеного в вимогах (розділ 5.2, 5.5).

7

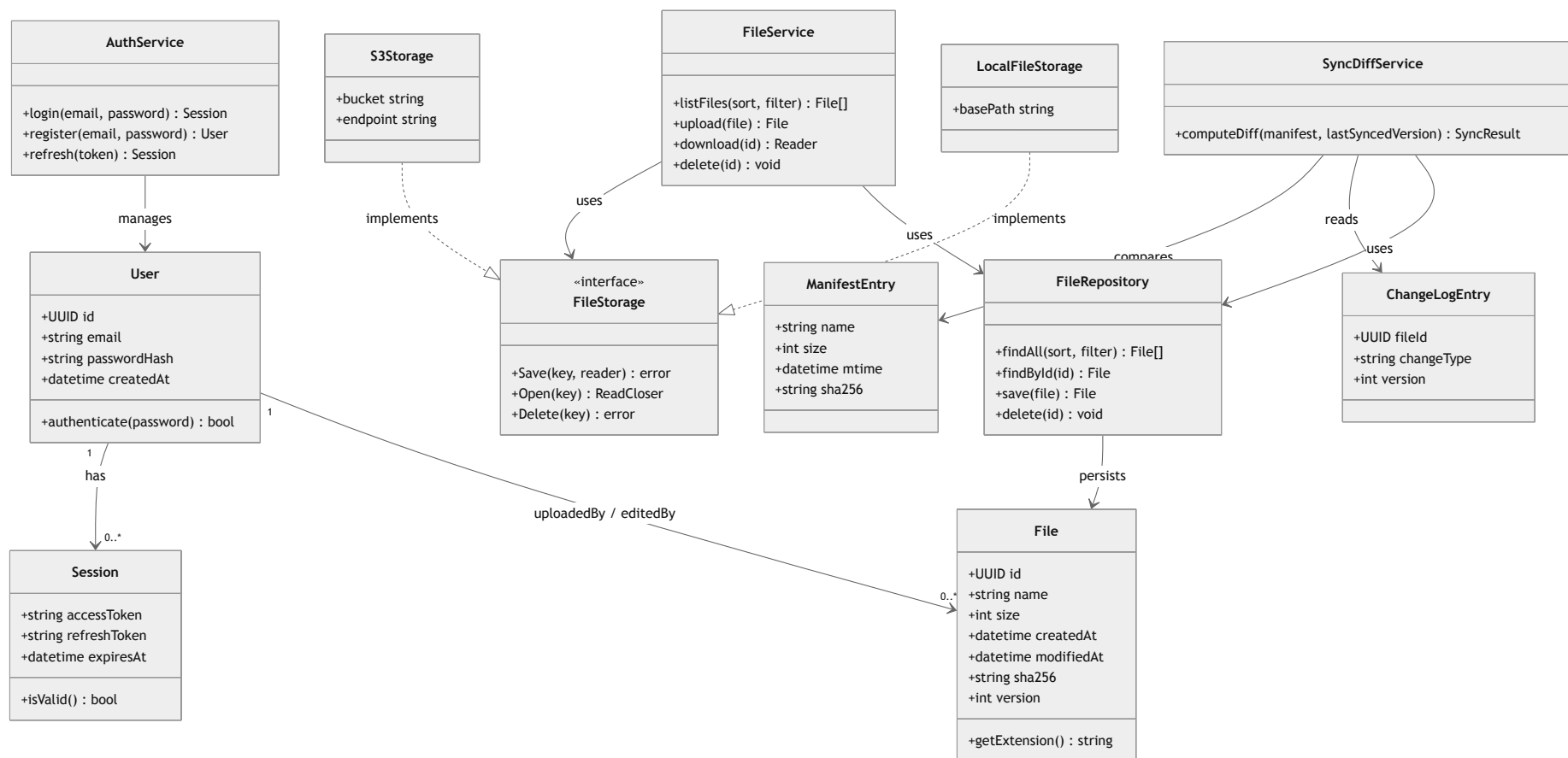


Рис. 2: Діаграма класів високого рівня (доменна модель)

2.3 Діаграма розгортання (загальна архітектура)

Рис. 3 відображає фізичну топологію системи: клієнтські пристрої (браузер, macOS), периферію Cloudflare (термінація TLS, публічний DNS) та домашній Ubuntu-сервер, що через Docker Compose підіймає три контейнери – `cloudflared` (вихідний тунель без відкриття портів), `api` (Go-бінарник) та `db` (PostgreSQL) – разом із постійним томом `pgdata` та `bind-mount` каталогом для файлового сховища.

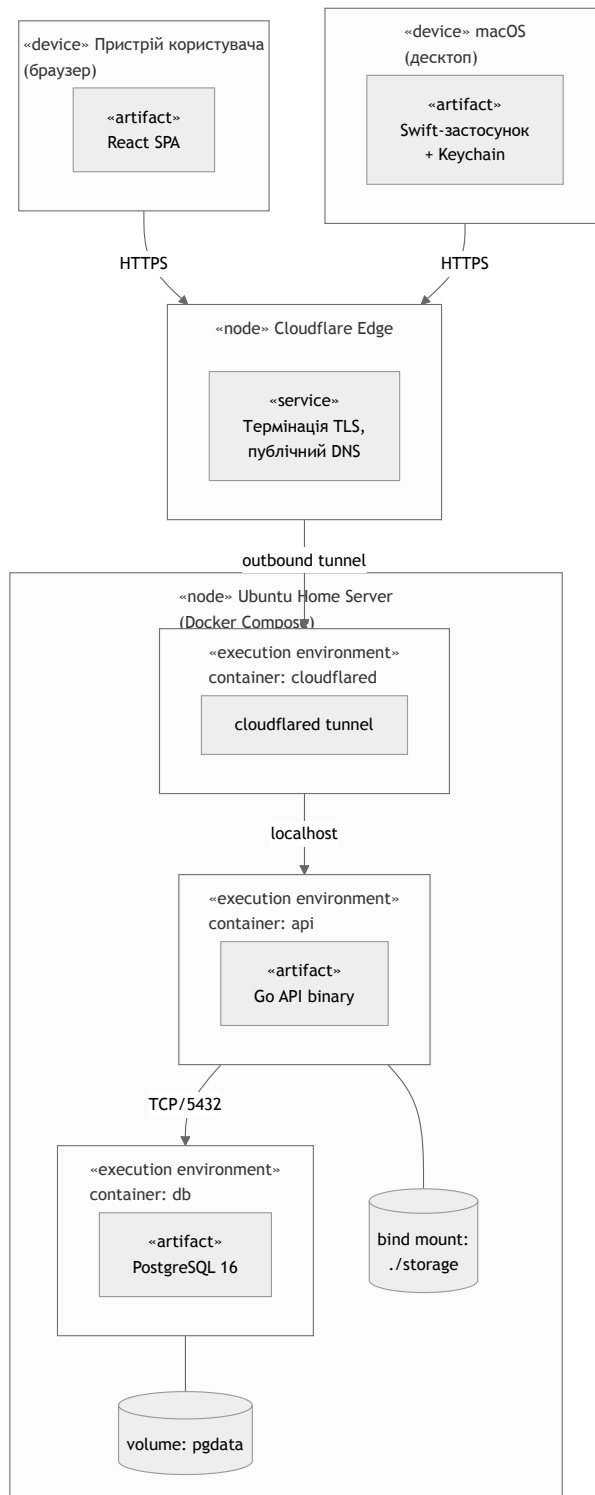


Рис. 3: Діаграма розгортання

3. Деталізоване проєктування

Деталізоване проєктування розкриває внутрішню структуру та поведінку окремих підсистем: класи, що беруть участь у реалізації конкретного прецедента (VOPC-діаграма), взаємодію об'єктів у часі (діаграми послі-

довності), поведінку файлу залежно від стану синхронізації (діаграма станів), структуру програмних компонентів (діаграма компонентів) та логіку бізнес-процесу синхронізації (діаграма активності).

3.1 VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”

Згідно з варіантом завдання, як прецедент для VOPC-діаграми (*View Of Participating Classes*) обрано “**Синхронізувати локальну теку з віддаленою**” – найскладніший і найбільш детально описаний у вимогах сценарій (розділ 6 вимог). Класи розподілено за стандартними стереотипами аналізу:

- «boundary» – елементи взаємодії з користувачем: спостерігач за текою (FolderWatcherUI), кнопка ручної синхронізації (SyncNowButtonUI), діалог розв’язання конфлікту (ConflictDialogUI);
- «control» – керуюча логіка: SyncController (клієнт), SyncDiffService (сервер, обчислення diff), ConflictResolver (застосування вибору користувача);
- «entity» – дані предметної області: ManifestEntry, SyncCursor, File, ChangeLogEntry.

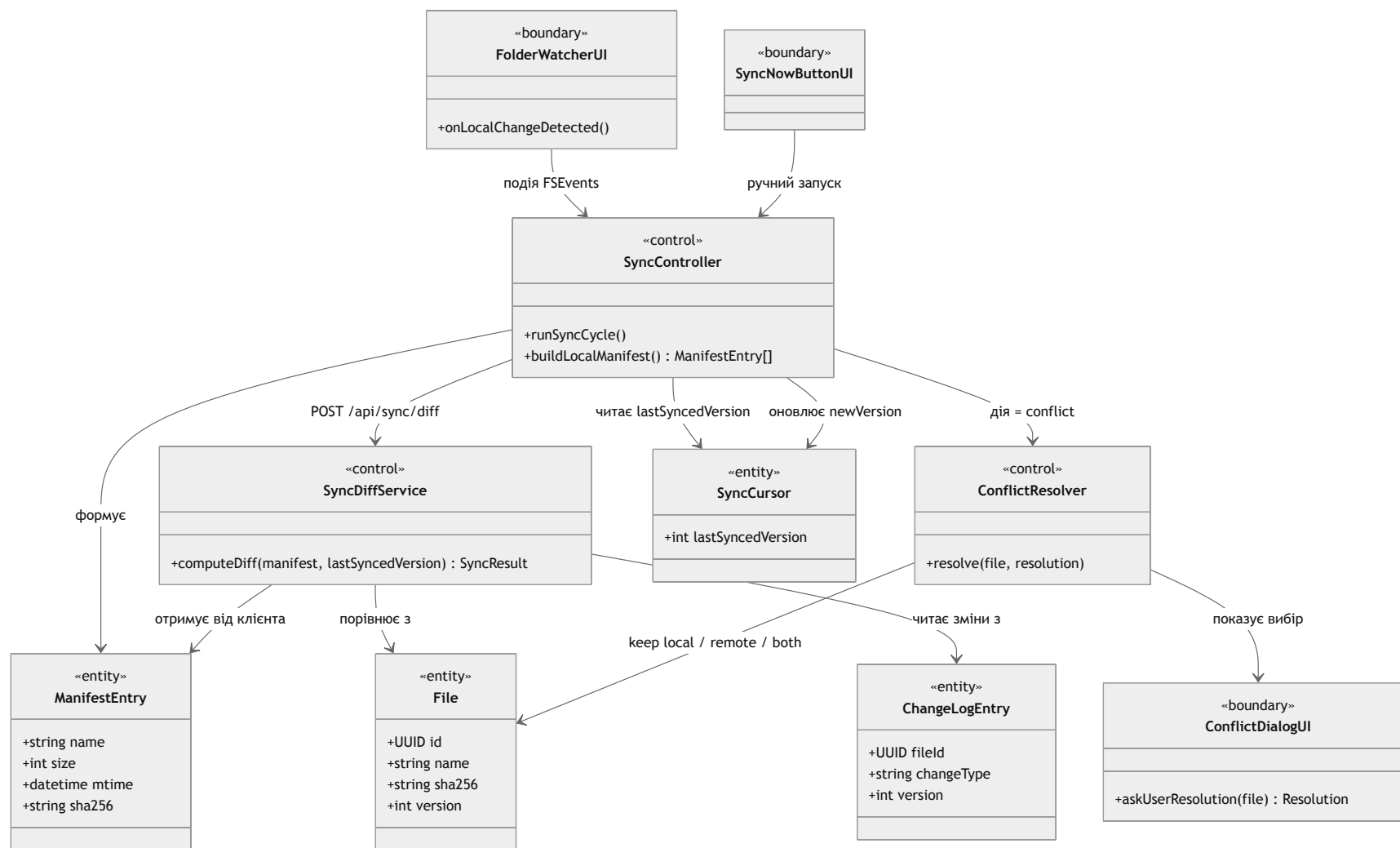


Рис. 4: VOPC-діаграма класів для прецедента "Синхронізувати локальну теку з віддаленою"

3.2 Діаграми взаємодії (Sequence Diagrams)

Наведено три діаграми послідовності, що ілюструють ключові сценарії взаємодії клієнта, Go API-сервера та бази даних.

Автентифікація

Рис. 5 показує послідовність перевірки облікових даних користувача та видачі пари токенів (access/refresh) відповідно до опису автентифікації у вимогах (розділ 1, 5.2).

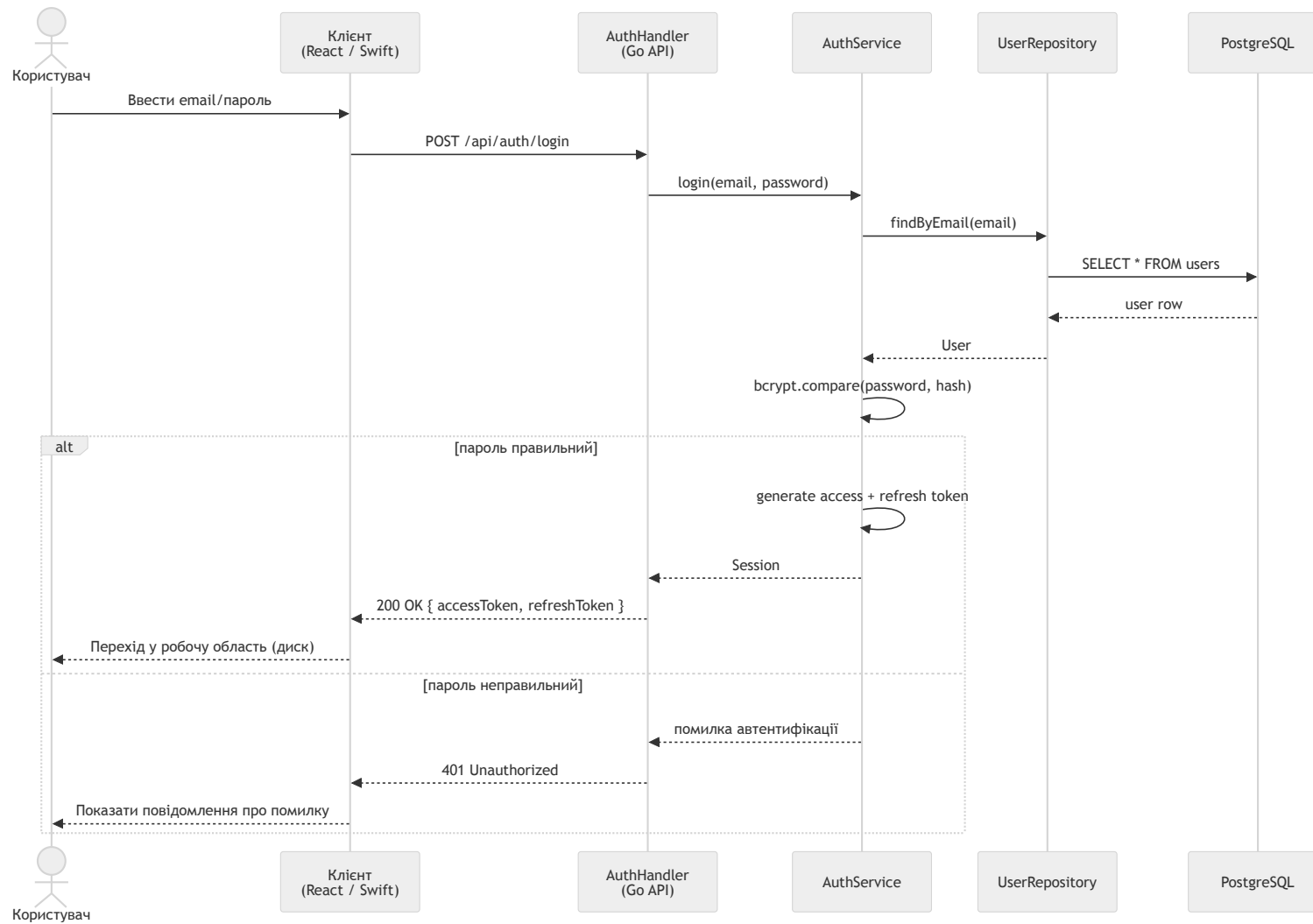


Рис. 5: Діаграма послідовності: автентифікація користувача

Вивантаження файлу

Рис. 6 ілюструє шлях запиту від клієнта через шар обробників (handler) до сервісного шару, який послідовно записує вміст файлу у сховище (FileStorage), а після підтвердження запису – метадані файлу у PostgreSQL через FileRepository.

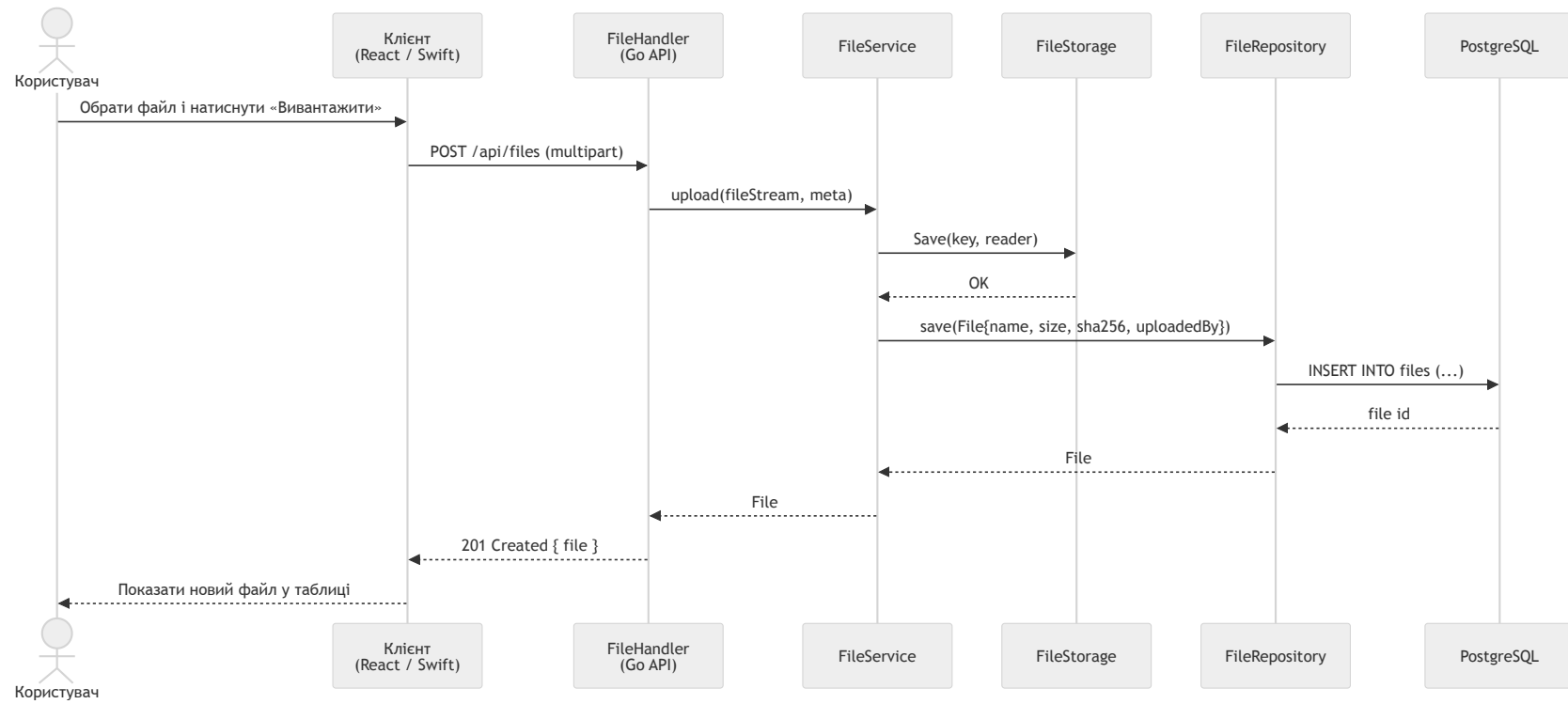


Рис. 6: Діаграма послідовності: вивантаження файлу на сервер

Синхронізація та розв'язання конфлікту

Рис. 7 деталізує протокол `POST /api/sync/diff` (розділ 6.6 вимог): клієнт надсилає локальний маніфест і `lastSyncedVersion`, сервер повертає перелік дій для кожного файлу, а конфліктні файли обробляються через запит вибору користувача (`keep local / keep remote / keep both`).

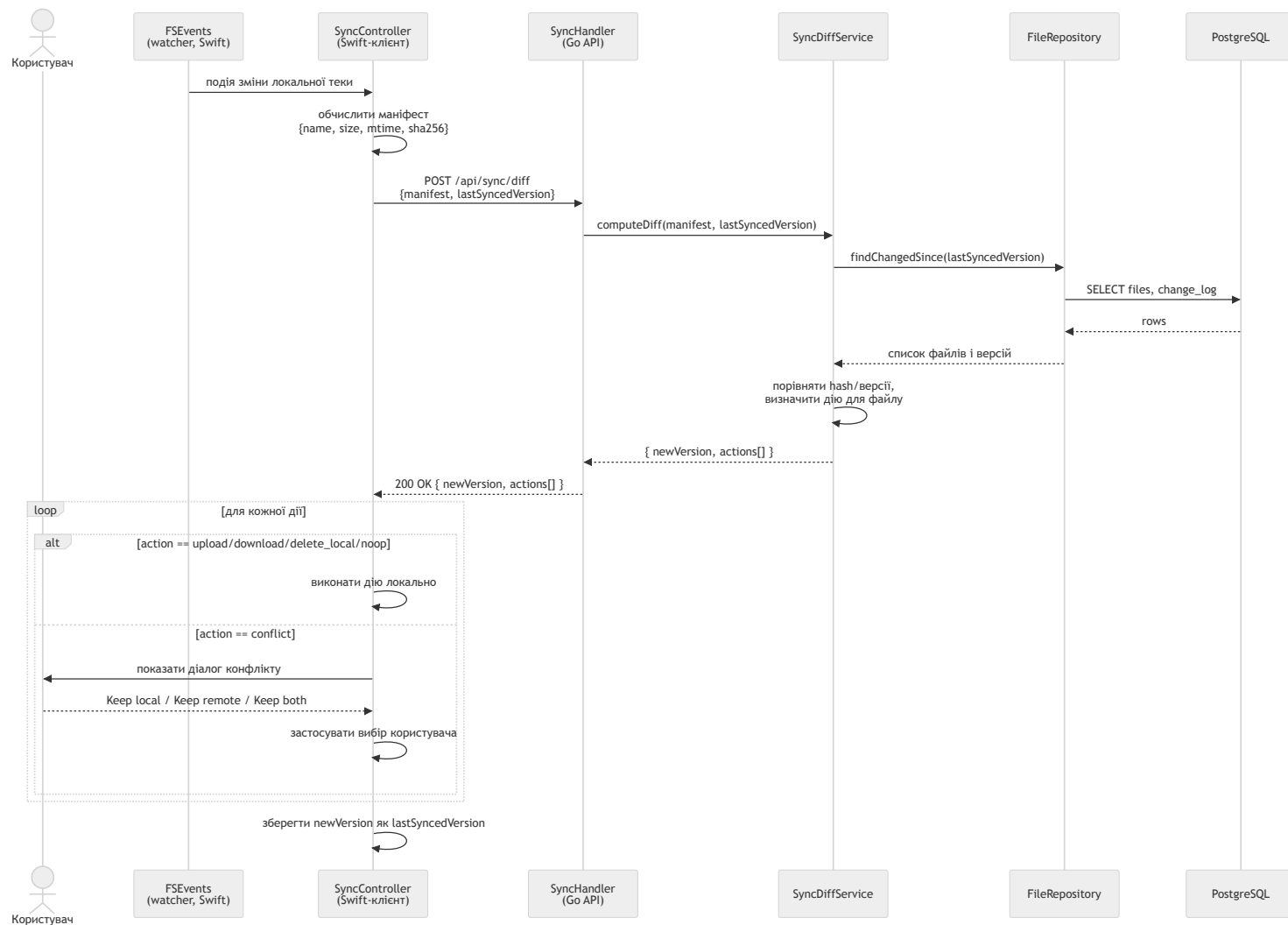


Рис. 7: Діаграма послідовності: синхронізація та розв'язання конфлікту

3.3 Діаграма станів

Рис. 8 описує поведінку сутності *File* залежно від стану синхронізації: від локального/віддаленого створення, через передачу (Uploading/Downloading), до синхронізованого стану, можливих локальних чи віддалених змін і, за одночасної зміни з обох боків, – конфлікту, що розв’язується користувачем. Діаграма безпосередньо реалізує чотири випадки порівняння маніфестів із розділу 6.2 вимог (upload / download / no-op / conflict) разом з обробкою видалень через версійний курсор (розділ 6.3).

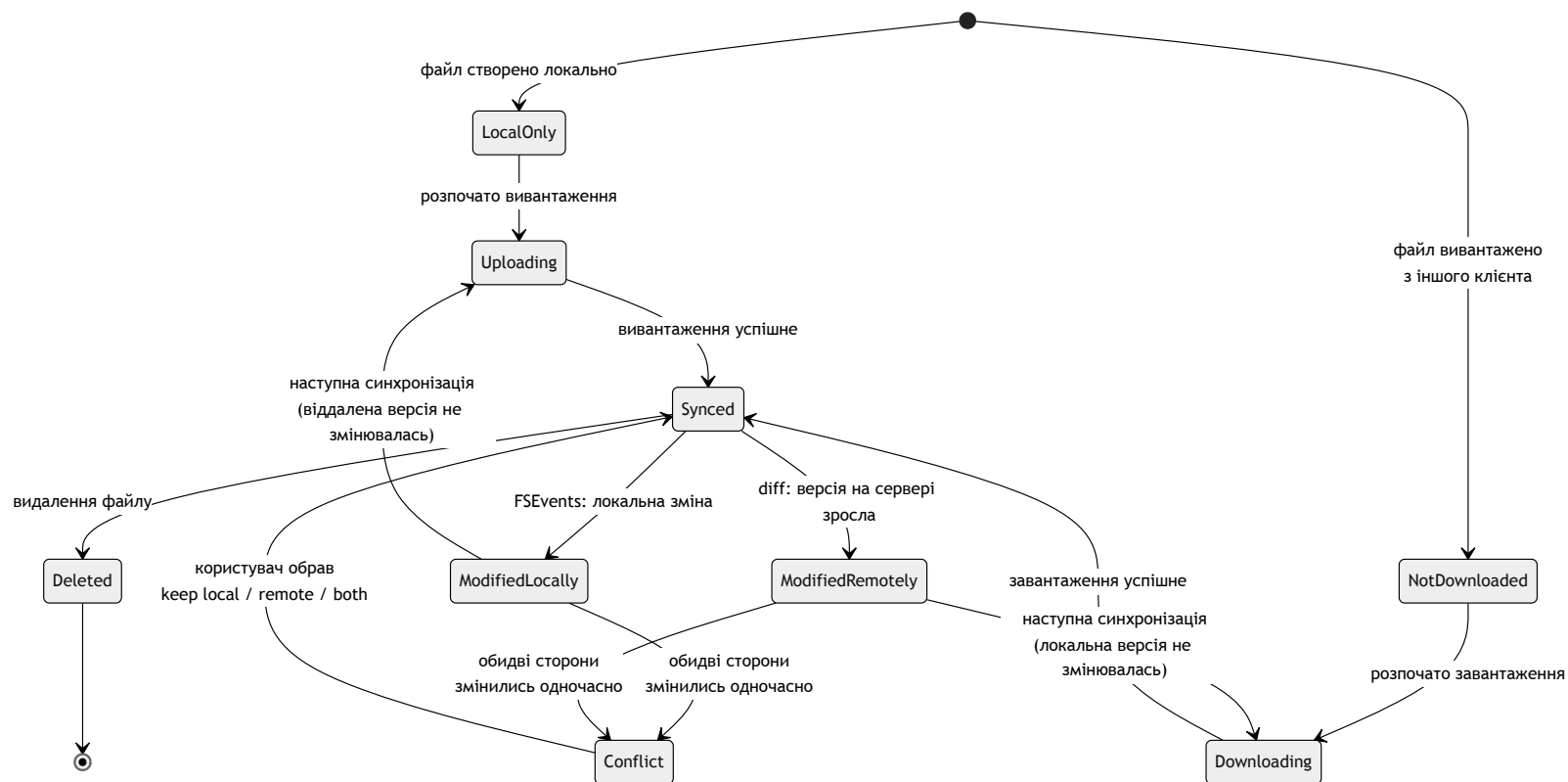


Рис. 8: Діаграма станів файлу в процесі синхронізації

3.4 Діаграма компонентів

Рис. 9 деталізує програмні компоненти системи та їх залежності: клієнтські застосунки, тунель cloudflared, а всередині Go API-сервера – шари Handler → Service → Repository, а також незалежний від конкретної реалізації інтерфейс FileStorage, що дозволяє замінити локальне сховище на S3-сумісне без зміни бізнес-логіки (розділ 5.2, 5.5 вимог).

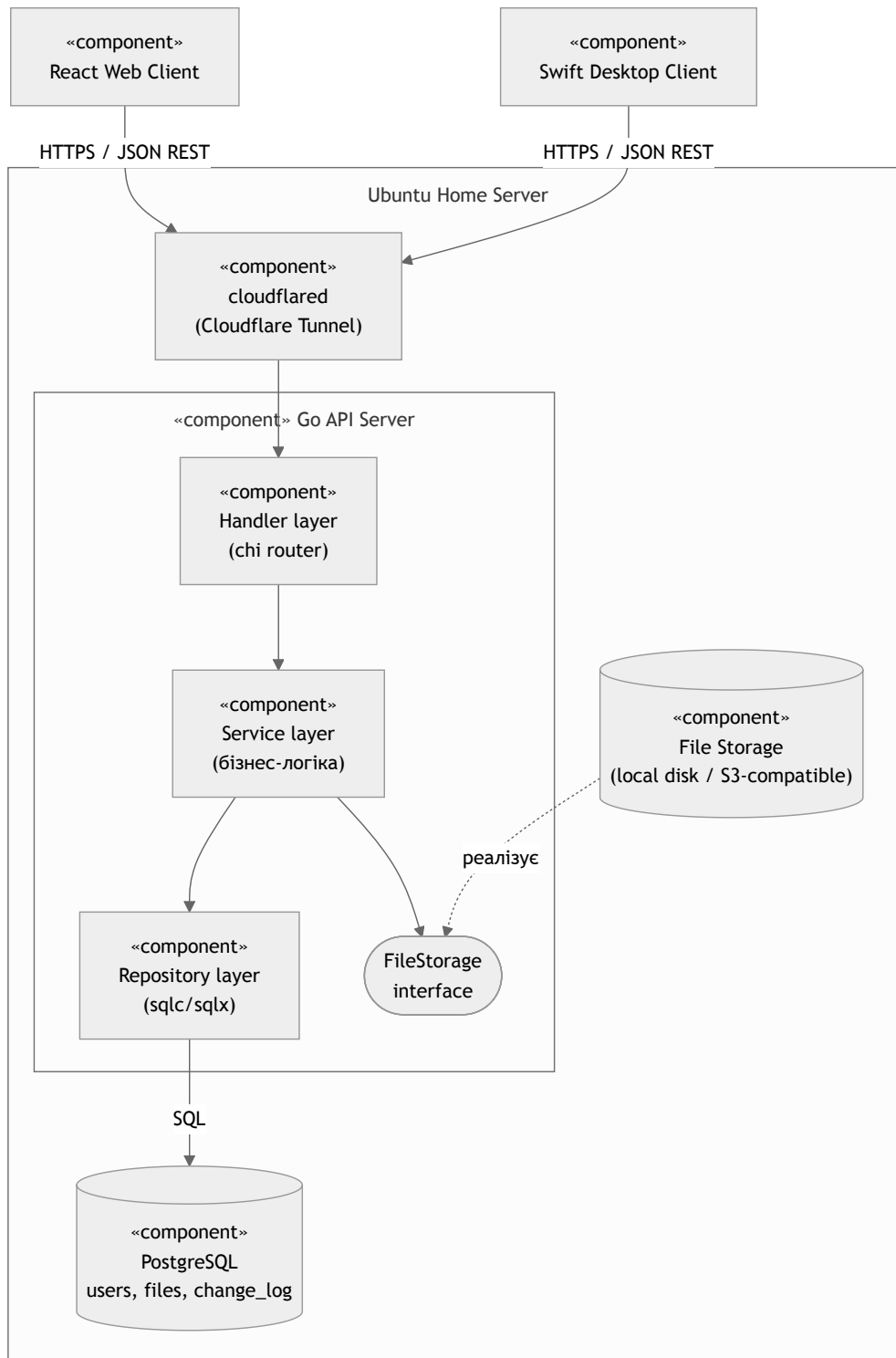


Рис. 9: Діаграма компонентів системи

3.5 Діаграма активності

Рис. 10 деталізує бізнес-логіку одного циклу синхронізації: обчислення локального маніфесту, надсилання його на сервер, визначення дії для кожного файлу (upload/download/no-op/conflict/delete_local), виконання дії або, у разі конфлікту, отримання рішення від користувача (keep local / keep remote / keep both) і, зрештою, збереження нової версії курсора синхронізації (розділ 6 вимог).

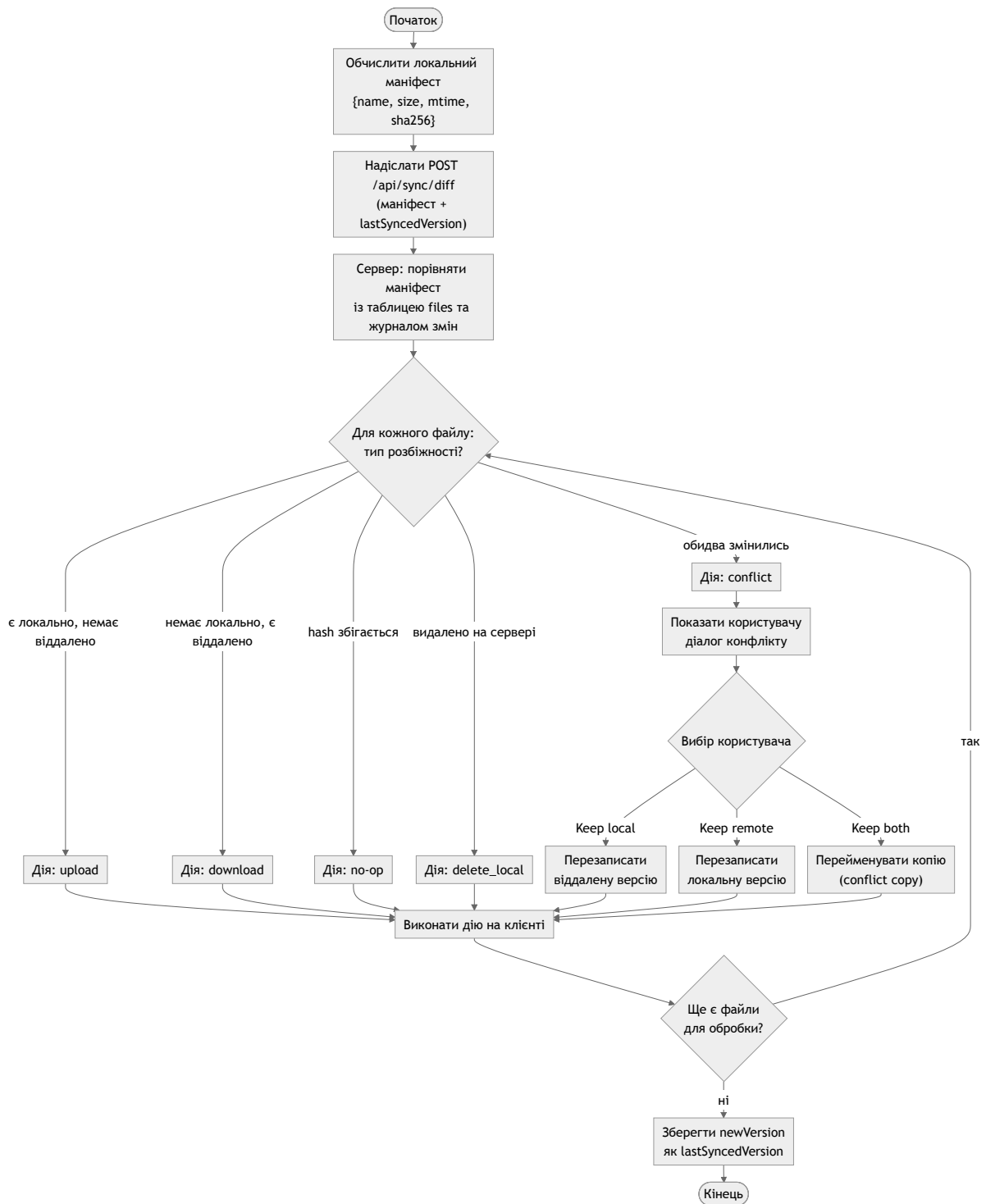


Рис. 10: Діаграма активності: цикл синхронізації

4. Висновки

Розроблений комплект діаграм послідовно розкриває систему Remote File Folder Client від бізнес-вимог до деталізованої структури і поведінки: діаграма прецедентів фіксує функціональні сценарії користувача; загальна діаграма класів і діаграма розгортання задають архітектурний каркас; VOPC-діаграма, діаграми послідовності, станів, компонентів та активності деталізують реалізацію найскладнішого сценарію – синхронізації файлів між локальною та віддаленою текою, включно з протоколом порівняння маніфестів та розв'язанням конфліктів редагування. Побудовані діаграми узгоджені між собою (спільні класи, сутності та дії) та слугують основою для подальшої розробки й тестування системи.